

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: K.Rama

Roll No.: A24126552087

Section: CSM-B

Date: 23-08-2026

1. TITLE

Design of an Interactive To-Do List Application using HTML, CSS, and JavaScript

2. AIM

To build a dynamic To-Do List web application that allows a user to add, complete, and delete tasks using HTML for structure, CSS for styling, and JavaScript for DOM manipulation and event handling.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how to link external CSS and JavaScript files to an HTML document.
- To dynamically create and append DOM elements using JavaScript.
- To handle user events such as click and keydown (Enter key).
- To toggle CSS classes for visual state changes (e.g., marking a task complete).
- To conditionally show or hide UI elements based on application state.

4. THEORY

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. Methods like `document.createElement()`, `appendChild()`, and `classList.add()/toggle()` allow new elements to be created and styled dynamically without reloading the page. Event listeners, added using `addEventListener()`, let JavaScript respond to user actions such as clicking a button or pressing a key. In this application, an input field and "Add Task" button let the user enter tasks, each task is rendered as a list item with "Complete" and "Delete" buttons, and CSS classes are toggled to reflect a task's completed state. Flexbox is used throughout the CSS for aligning the input section and each task row.

5. ALGORITHM / PROCEDURE

1. Create `index.html` with a container `div` holding a heading, an input field, an 'Add Task' button, an empty `<ul>` for tasks, and an empty-state message.
2. Link an external `style.css` file and an external `script.js` file.
3. In `style.css`, style the container, input section (using Flexbox), task items, and Complete/Delete buttons, including hover and completed-state styles.
4. In `script.js`, select the input field, button, task list, and empty message elements using `getElementById()`.

5. Write an `addTask()` function that reads the input value, validates it is not empty, and creates a new `<li>` containing a text span, a Complete button, and a Delete button.
6. Append the new list item to the task list and clear the input field.
7. Attach a click event listener to the Complete button to toggle a 'completed' CSS class on the task text.
8. Attach a click event listener to the Delete button to remove the task item and update the empty-state message.
9. Attach a click event listener on the 'Add Task' button and a keydown listener on the input field (for the Enter key) to call `addTask()`.
10. Call `updateEmptyMessage()` on page load and after every add/delete action to show or hide the 'No tasks available.' message.
11. Test the application in a browser: add tasks, mark tasks complete, and delete tasks.

6. CODE EXPLANATION

The application is split into three files. `index.html` defines the page structure: a container with a heading, an input field with id "taskInput", an "Add Task" button, an empty unordered list with id "taskList", and a paragraph with id "emptyMessage" that is shown only when there are no tasks. `style.css` uses Flexbox to center the container on the page and to lay out the input section and each task row horizontally; it also styles the Complete button green, the Delete button red, and applies a strikethrough via the `.completed` class. `script.js` contains the application logic: `addTask()` builds a new `<li>` element (with a text `<span>`, a Complete `<button>`, and a Delete `<button>`) using `document.createElement()` and `appendChild()`, and attaches its own click listeners for completing and deleting that specific task. Event listeners on the 'Add Task' button and the Enter key both call `addTask()`, and `updateEmptyMessage()` toggles the visibility of the 'No tasks available.' message based on how many `<li>` elements exist in the list.

7. PROGRAM

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>My To-Do List</title>

  <!-- External CSS -->
  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <h1>My To-Do List</h1>

    <!-- Input section -->
    <div class="input-section">
```

```

        <input
            type="text"
            id="taskInput"
            placeholder="Enter a task....."
        >

        <button id="addTaskBtn">Add Task</button>

    </div>

    <!-- Task list -->
    <ul id="taskList"></ul>

    <!-- Empty list message -->
    <p id="emptyMessage">No tasks available.</p>

</div>

<!-- JavaScript -->
<script src="script.js"></script>

</body>
</html>

```

style.css

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    min-height: 100vh;

    display: flex;
    justify-content: center;
    align-items: center;
}

.container {
    width: 500px;
    background-color: white;
    padding: 30px;
    border-radius: 10px;

    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);
}

h1 {

```

```
    text-align: center;
    margin-bottom: 25px;
}

.input-section {
    display: flex;
    gap: 10px;
}

#taskInput {
    flex: 1;
    padding: 12px;

    border: 1px solid #ccc;
    border-radius: 5px;

    font-size: 16px;
}

#addTaskBtn {
    padding: 12px 18px;

    border: none;
    border-radius: 5px;

    background-color: #333;
    color: white;

    cursor: pointer;
}

#addTaskBtn:hover {
    background-color: #555;
}

#taskList {
    list-style: none;
    margin-top: 20px;
}

.task-item {
    display: flex;
    align-items: center;
    justify-content: space-between;

    padding: 12px;

    margin-bottom: 10px;

    background-color: #f5f5f5;

    border-radius: 5px;
}
```

```

}

.task-text {
  flex: 1;
  font-size: 16px;
}

.task-item button {
  margin-left: 8px;

  padding: 8px 12px;

  border: none;
  border-radius: 4px;

  cursor: pointer;
}

.complete-btn {
  background-color: #4CAF50;
  color: white;
}

.delete-btn {
  background-color: #f44336;
  color: white;
}

.completed {
  text-decoration: line-through;
  color: gray;
}

#emptyMessage {
  text-align: center;
  margin-top: 20px;

  color: gray;
}

```

script.js

```

// Select HTML elements
const taskInput = document.getElementById("taskInput");
const addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

// Function to update empty message
function updateEmptyMessage() {

  if (taskList.children.length === 0) {
    emptyMessage.style.display = "block";
  }
}

```

```

    } else {
        emptyMessage.style.display = "none";
    }
}
// Function to add a new task
function addTask() {

    // Get the value entered by the user

    const taskText = taskInput.value.trim();

    // Check if input is empty
    if (taskText === "") {
        alert("Please enter a task.");
        return;
    }

    // Create a new list item

    const li = document.createElement("li");
    li.classList.add("task-item");

    // Create span for task text

    const span = document.createElement("span");
    span.classList.add("task-text");
    span.textContent = taskText;

    // Create Complete button

    const completeBtn = document.createElement("button");
    completeBtn.textContent = "Complete";
    completeBtn.classList.add("complete-btn");

    // Create Delete button

    const deleteBtn = document.createElement("button");

```

```
deleteBtn.textContent = "Delete";

deleteBtn.classList.add("delete-btn");


// Add elements to list item
li.appendChild(span);
li.appendChild(completeBtn);
li.appendChild(deleteBtn);


// Add list item to task list
taskList.appendChild(li);


// Clear input box
taskInput.value = "";


// Put cursor back in input box
taskInput.focus();


// Complete button event listener
completeBtn.addEventListener("click", function () {
    span.classList.toggle("completed");
});


// Delete button event listener
deleteBtn.addEventListener("click", function () {
    li.remove();
    updateEmptyMessage();
});


// Update empty message
updateEmptyMessage();
```

```

}

// Add click event to Add Task button
addTaskBtn.addEventListener("click", addTask);

// Allow Enter key to add a task
taskInput.addEventListener("keydown", function (event) {
    if (event.key === "Enter") {
        addTask();
    }
});

// Display empty message when page loads
updateEmptyMessage();

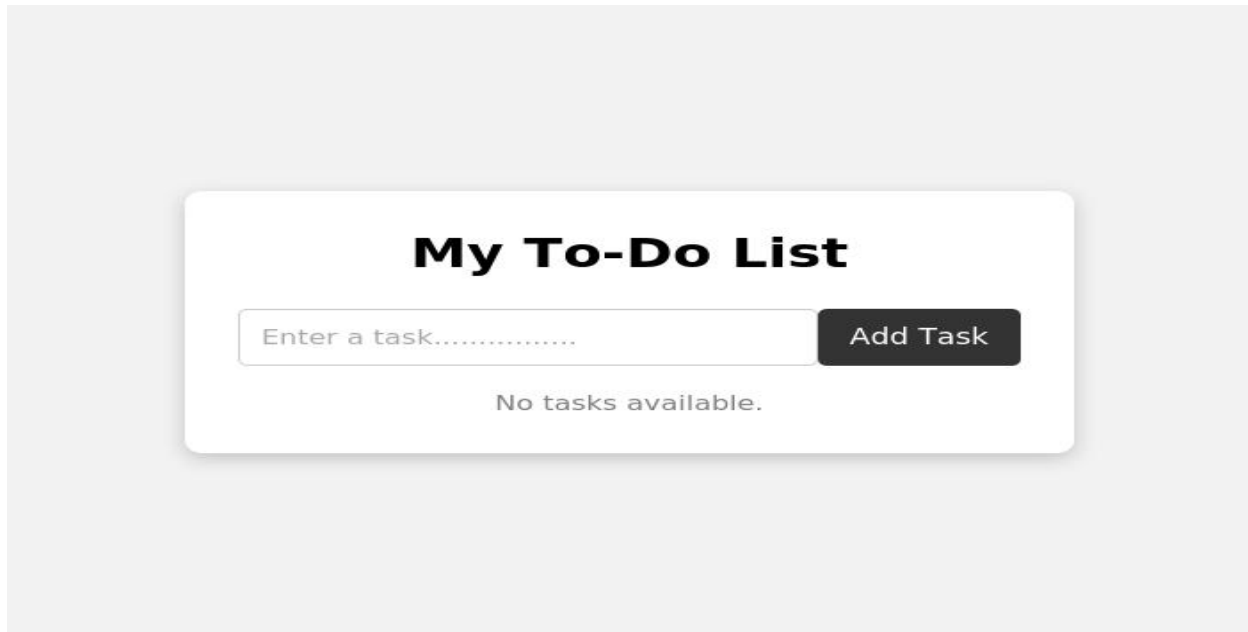
```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load page with no tasks	"No tasks available." message shown	Empty message displayed correctly	Pass
TC-02	Enter task and click Add Task	New task appears as a list item with Complete/Delete buttons	Task added to the list, input field cleared	Pass
TC-03	Click Add Task with empty input	Alert "Please enter a task." is shown	Alert displayed, no task added	Pass
TC-04	Click Complete button on a task	Task text gets a strikethrough style	Task text shown with line-through, gray color	Pass
TC-05	Click Delete button on a task	Task is removed from the list	Task removed; empty message reappears if list is empty	Pass
TC-06	Press Enter key in input field	Task is added, same as clicking Add Task	Task added successfully via Enter key	Pass

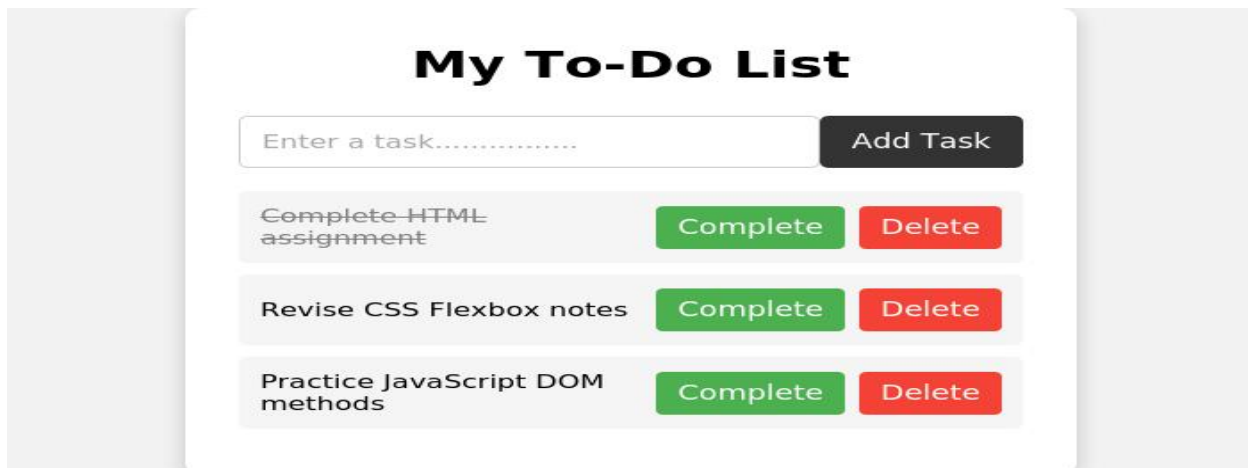
9. OUTPUT

Initial page load (no tasks added):



The screenshot shows a web application titled "My To-Do List". It features a text input field with the placeholder text "Enter a task....." and a dark grey "Add Task" button. Below the input field, the text "No tasks available." is displayed. The entire interface is centered within a light grey background.

After adding tasks and marking one as complete:



The screenshot shows the "My To-Do List" application with three tasks listed. Each task is in a light grey box and includes a green "Complete" button and a red "Delete" button. The first task, "Complete HTML assignment", has the word "Complete" crossed out. The other two tasks are "Revise CSS Flexbox notes" and "Practice JavaScript DOM methods". The input field and "Add Task" button are still present at the top.

Task	Complete	Delete
Complete HTML assignment	Complete	Delete
Revise CSS Flexbox notes	Complete	Delete
Practice JavaScript DOM methods	Complete	Delete

10. RESULT

Thus, an interactive To-Do List web application was successfully designed using HTML, CSS, and JavaScript. The application allows users to add, complete, and delete tasks dynamically, and the output was verified in a browser.